AURORA SINGER

LINK DO REPOSITÓRIO DO GITHUB COM O NOTEBOOK:
https://github.com/Projeto-Aurora-Singer/Projeto-1---Lan-amento-Aurora-Singer.git

RELATÓRIO ELABORADO EM CONJUNTO POR:

- Mayara Luisa Vicente Rosa - RM: 571955
- Gabriel Coutinho Barcelos - RM: 573211
- Pedro Henrique de Souza Elias - RM: 570487
- Gabriel Luis de Lima Ramos - RM:568984

1. ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA INTERPRETAR DADOS REFERENTES A:

A telemetria corresponde ao conjunto de dados coletados por sensores responsáveis por monitorar continuamente o estado de um sistema. Esses dados são fundamentais para identificar possíveis falhas e apoiar a tomada de decisões antes da execução de operações críticas.

O dataset utilizado está disponível no repositório da equipe no GitHub, podendo ser acessado por meio do seguinte link: https://raw.githubusercontent.com/Projeto-Aurora-Singer/Projeto-1---Lan-amento-Aurora-Singer/refs/heads/main/dataset_telemetria_sintetico_anomalias_ptbr.csv

Neste contexto, foi utilizado um dataset sintético de telemetria, contendo variáveis relacionadas à temperatura, energia, integridade estrutural e funcionamento de módulos essenciais. Os dados são fictícios e foram gerados com o apoio de uma ferramenta de Inteligência Artificial, com o objetivo de simular o comportamento de um sistema monitorado por telemetria.

O processo de geração dos dados, bem como o uso da Inteligência Artificial, será descrito com mais detalhes no tópico 5 deste relatório.

A seguir são descritas as principais variáveis monitoradas.

1.1 Temperatura interna e externa

As variáveis internal_temp_c e external_temp_c representam, respectivamente, a temperatura interna do sistema e a temperatura do ambiente externo.

A temperatura interna é um indicador importante do funcionamento dos componentes eletrónicos e do sistema de baterias. Valores dentro da faixa operacional indicam que os componentes estão funcionando em condições

adequadas. Temperaturas muito elevadas podem indicar sobrecarga dos sistemas, falhas no sistema de resfriamento ou risco de superaquecimento.

A temperatura externa também pode influenciar o comportamento do sistema, especialmente no desempenho térmico dos componentes e na eficiência energética.

1.2 Integridade estrutural

A variável structural_integrity indica o estado da estrutura do sistema e é representada de forma binária:

- **1** – estrutura íntegra
- **0** – falha estrutural

A integridade estrutural é um fator crítico para a segurança da operação. Uma falha estrutural pode indicar danos físicos ou comprometimento da estabilidade do sistema, podendo resultar em risco operacional elevado.

1.3 Níveis de energia

Os níveis de energia são representados principalmente pela variável battery_soc_percent, que indica o estado de carga da bateria em porcentagem.

Esse valor representa a quantidade de energia disponível no sistema em relação à capacidade total da bateria. Níveis adequados de energia são essenciais para garantir o funcionamento contínuo dos componentes e módulos críticos do sistema.

Valores baixos de carga podem comprometer a autonomia do sistema e dificultar a execução de operações que exigem maior consumo energético.

1.4 Pressão dos tanques

A variável tank_pressure_bar representa a pressão interna dos tanques do sistema.

A pressão adequada é fundamental para garantir estabilidade e funcionamento correto dos sistemas associados ao armazenamento. Valores fora da faixa operacional podem indicar vazamentos, falhas em válvulas ou risco de sobrepressão, situações que podem comprometer a segurança da operação.

1.5 Status dos módulos críticos

As variáveis critical_modules_status e telemetry_link_status indicam o estado de funcionamento de módulos essenciais do sistema.

Essas variáveis são representadas de forma binária:

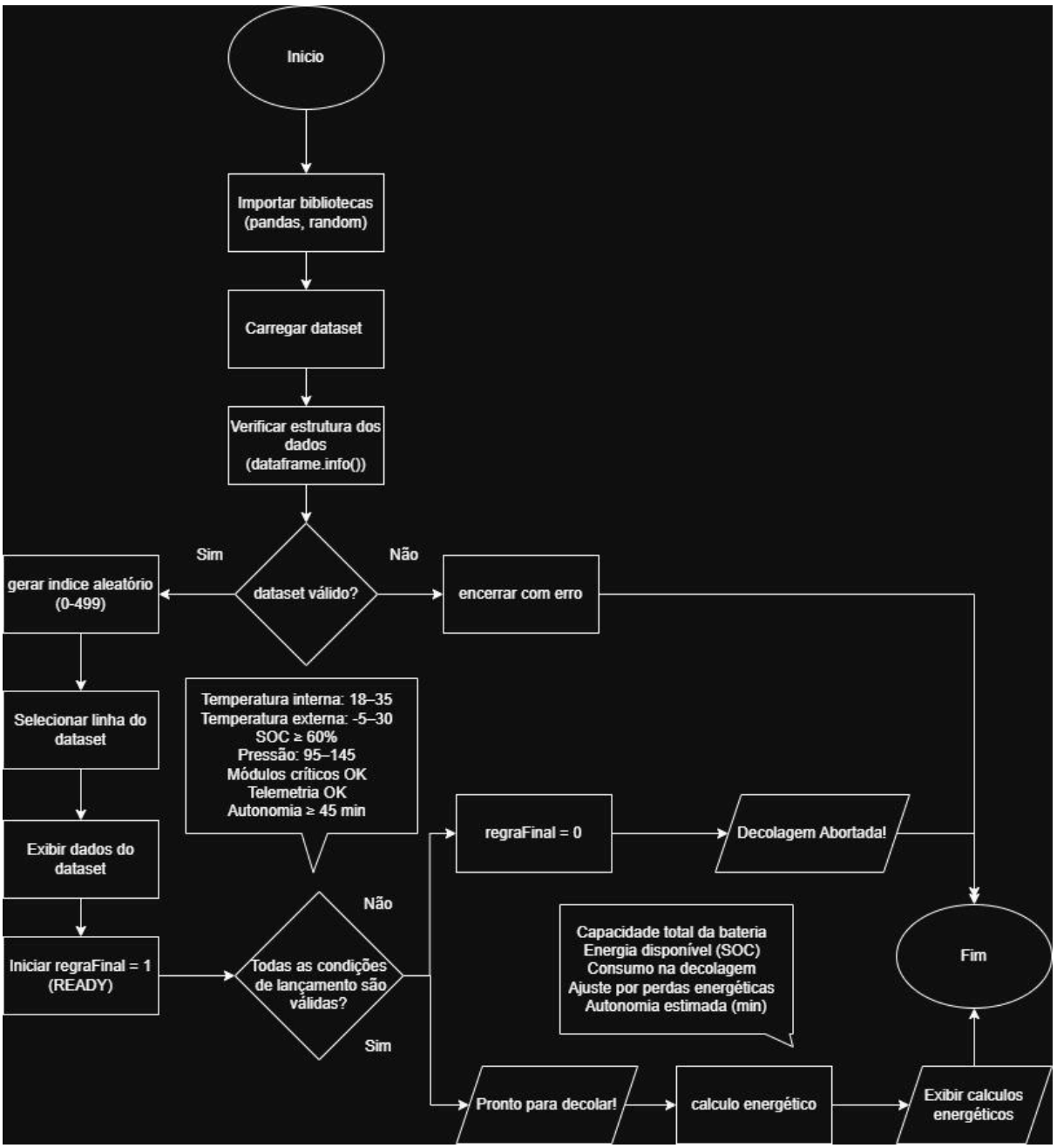
- **1** – funcionamento normal
- **0** – falha detectada

Os módulos críticos são responsáveis pelo controle, monitoramento e comunicação dos dados de telemetria. Caso algum desses módulos apresente falha, o sistema pode perder a capacidade de monitorar corretamente suas condições operacionais, aumentando o risco de falhas não identificadas durante a operação.

2. ALGORITMO DE VERIFICAÇÃO:

O algoritmo de verificação tem como objetivo avaliar, com base nos dados de telemetria, se a nave está apta para decolagem. A decisão final é baseada em regras de segurança previamente definidas.

Abaixo encontra-se o fluxograma desenvolvido para facilitar a compreensão do processo:



3. SCRIPT EM PYTHON IMPLEMENTAR A LÓGICA DO ALGORITMO EM PYTHON, SIMULANDO:

O script em Python utilizado foi:

```
import pandas as pd
import random as rd

# Leitura do dataset utilizando separador ";"
DataFrame =
pd.read_csv('https://raw.githubusercontent.com/Projeto-Aurora-Singer/Projeto-1---La
```

n-amento-Aurora-Singer/refs/heads/main/dataset_telemetria_sintetico_anomalias_ptbr.csv', sep=';')

 # Mostra as primeiras linhas do dataset
DataFrame.head()

	timestamp	internal_temp_c	external_temp_c	battery_voltage_v	battery_current_a	battery_soc_percent	battery_capacity_ah	energy_available_kwh
0	2026-03-18 20:00:00	18,89	-1,95	48,44	30,77	96,05	81,53	3,7933
1	2026-03-18 20:01:00	23,73	6,68	47,47	20,17	22,99	83,5	0,9113
2	2026-03-18 20:02:00	33,39	-1,75	46,85	99,01	60,85	116,45	3,3198
3	2026-03-18 20:03:00	23,84	23,05	47,24	80,89	81,08	112,37	4,304
4	2026-03-18 20:04:00	29,64	17,34	50,0	109,01	79,45	106,74	4,2402

(A visualização acima representa apenas uma amostra do dataset, sendo possível observar todas as variáveis ao executar o código.)

 # Exibe informações gerais (tipos de dados e valores não nulos)
DataFrame.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 17 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   timestamp               500 non-null    object
 1   internal_temp_c         500 non-null    object
 2   external_temp_c         500 non-null    object
 3   battery_voltage_v       500 non-null    object
 4   battery_current_a       500 non-null    object
 5   battery_soc_percent     500 non-null    object
 6   battery_capacity_ah     500 non-null    object
 7   energy_available_kwh    500 non-null    object
 8   power_load_kw           500 non-null    object
 9   energy_loss_percent     500 non-null    object
 10  tank_pressure_bar       500 non-null    object
 11  structural_integrity    500 non-null    int64
 12  critical_modules_status 500 non-null    int64
 13  telemetry_link_status   500 non-null    int64
 14  estimated_autonomy_min  500 non-null    object
 15  launch_decision         500 non-null    object
 16  anomalia_inserida       500 non-null    object
dtypes: int64(3), object(14)
memory usage: 66.5+ KB
```

 # Seleciona uma linha aleatória do dataset para simular um lançamento
 # O índice vai de 0 até 499
lancamento = rd.randint(0, 499)

 # Extrai os dados da linha selecionada
infosLancamento = DataFrame.iloc[lancamento]

```
# Exibe os dados do lançamento selecionado
print(infosLancamento)

# Variável de controle da decisão final (1 = READY, 0 = ABORT)
regraFinal = 1
```

```
timestamp                  2026-03-18 22:38:00
internal_temp_c                          33,65
external_temp_c                           4,51
battery_voltage_v                        48,24
battery_current_a                        77,82
battery_soc_percent                      76,84
battery_capacity_ah                      98,64
energy_available_kwh                    3,6563
power_load_kw                            15,36
energy_loss_percent                       2,11
tank_pressure_bar                       116,17
structural_integrity                         1
critical_modules_status                      1
telemetry_link_status                        1
estimated_autonomy_min                   51,72
launch_decision                          READY
anomalia_inserida                          nao
Name: 158, dtype: object
```

(Se trata de um exemplo, já que possui o random e aparecerá números diversos toda vez)

```
# Verifica temperatura interna dentro do limite seguro
if not 18 <= (float(infosLancamento["internal_temp_c"].replace(',', '.'))) <= 35:
    regraFinal = 0

# Verifica temperatura externa
if not -5 <= (float(infosLancamento["external_temp_c"].replace(',', '.'))) <= 30:
    regraFinal = 0

# Verifica estado de carga da bateria (SOC)
if not 60 <= (float(infosLancamento["battery_soc_percent"].replace(',', '.'))):
    regraFinal = 0

# Verifica pressão do tanque
if not 95 <= (float(infosLancamento["tank_pressure_bar"].replace(',', '.'))) <= 145:
    regraFinal = 0

# Verifica se módulos críticos estão operacionais
if 0 == infosLancamento['critical_modules_status']:
    regraFinal = 0

# Verifica status da telemetria
if infosLancamento["telemetry_link_status"] != 1:
```

```
    regraFinal = 0

# Verifica autonomia mínima necessária
if not (float(infosLancamento["estimated_autonomy_min"].replace(',', '.'))) >= 45:
    regraFinal = 0

# Tempo fixo necessário para decolagem
tempo_decolagem_min = 8

# Verifica se há autonomia suficiente para a decolagem
if (float(infosLancamento["estimated_autonomy_min"].replace(',', '.'))) <
tempo_decolagem_min:
    regraFinal = 0

# Resultado final da decisão
if regraFinal == 0:
    print("DECOLAGEM ABORTADA")
else:
    print("PRONTO PARA DECOLAR")
```

```
PRONTO PARA DECOLAR
```

4. ANÁLISE ENERGÉTICA CALCULAR AUTONOMIA INICIAL CONSIDERANDO:

A análise energética tem como objetivo avaliar se a nave possui autonomia suficiente para executar a decolagem com segurança, considerando consumo e perdas energéticas.

Para isso, foram utilizadas as seguintes fórmulas:

- Capacidade total da bateria (kWh): - ***capacidade_total_kwh = (tensão * capacidade_ah) / 1000***
- Energia disponível: - energia_disponivel = ***capacidade_total_kwh * (SOC / 100)***
- Consumo estimado na decolagem: - ***consumo_decolagem = potência * (tempo_decolagem / 60)***
- Consumo real considerando perdas: - ***consumo_real = consumo_decolagem * (1 + perdas / 100)***
- Autonomia estimada: ***- autonomia = (energia_disponivel / (potência * (1 + perdas / 100))) * 60***

O script em foi utilizado em Python e os seus seguintes resultados foram:

```
# Calcula a capacidade total da bateria em kWh
capacidadeTotalKwh = float(infosLancamento['battery_voltage_v'].replace(',', '.')) * float(infosLancamento["battery_capacity_ah"].replace(',', '.')) / 1000
print("O total de energia que a bateria pode armazenar é:", capacidadeTotalKwh)
```

```
O total de energia que a bateria pode armazenar é: 5.348732
```

```
# Calcula a energia disponível com base no estado de carga (SOC)
energyAvailableKwh = capacidadeTotalKwh * float(infosLancamento["battery_soc_percent"].replace(',', '.')) / 100
print("A quantidade de energia disponível atualmente na bateria é (kWh):", energyAvailableKwh)
```

```
A quantidade de energia disponível atualmente na bateria é (kWh): 5.049203008
```

```
# Calcula o consumo estimado durante a decolagem
takeOffConsumptionKwh = float(infosLancamento["power_load_kw"].replace(',', '.')) * (float(infosLancamento["estimated_autonomy_min"].replace(',', '.')) / 60)
print("O consumo estimado de energia durante a decolagem é (kWh):",takeOffConsumptionKwh)
```

```
O consumo estimado de energia durante a decolagem é (kWh): 28.466280000000005
```

```
# Ajusta o consumo considerando perdas energéticas
realTakeoffConsumptionKwh = takeOffConsumptionKwh * (1 + (float(infosLancamento['energy_loss_percent'].replace(',', '.'))) / 100)
print("O consumo real de energia considerando perdas é (kWh):", realTakeoffConsumptionKwh)
```

```
O consumo real de energia considerando perdas é (kWh): 29.755802484000004
```

```
# Calcula a autonomia estimada do sistema (em minutos)
estimatedAutonomyMin = (float(infosLancamento["energy_available_kwh"].replace(',', '.')) / (float(infosLancamento["power_load_kw"].replace(',', '.')) * (1 + (float(infosLancamento["energy_loss_percent"].replace(',', '.')) / 100)))) * 60
```

```
print("A autonomia estimada do sistema é (minutos):",
estimatedAutonomyMin)
```

```
A autonomia estimada do sistema é (minutos): 15.04792405584648
```

O sistema desenvolvido permite simular decisões de lançamento com base em dados de telemetria e regras pré-definidas.

Além disso, a análise energética complementa a validação ao garantir que a nave possui autonomia suficiente para executar a decolagem com segurança.

5. ANÁLISE ASSISTIDA POR IA:

Após a geração do dataset sintético de telemetria, foi utilizada uma ferramenta de Inteligência Artificial, o ChatGPT (modelo GPT-5.3), desenvolvido pela OpenAI, com o objetivo de apoiar a interpretação dos dados e identificar possíveis problemas operacionais no sistema simulado.

O objetivo dessa etapa foi utilizar a IA como apoio analítico tanto na criação do dataset quanto na sua classificação, permitindo a identificação de possíveis anomalias e a sugestão de riscos operacionais.

Para isso, foi utilizado o seguinte prompt:

> "*Crie um dataset sintético em formato CSV para simulação de telemetria de um sistema energético.*
> *O dataset deve conter 500 linhas com timestamps sequenciais, representando leituras ao longo do tempo.*
> *Colunas obrigatórias:*
> *timestamp, internal_temp_c, external_temp_c, battery_voltage_v, battery_current_a, battery_soc_percent, battery_capacity_ah, energy_available_kwh, power_load_kw, energy_loss_percent, tank_pressure_bar, structural_integrity, critical_modules_status, telemetry_link_status, estimated_autonomy_min, launch_decision.*
>
> *Limites de segurança:*
> *internal_temp_c: entre 18 e 35*
> *external_temp_c: entre -5 e 30*
> *battery_soc_percent: mínimo de 60*
> *tank_pressure_bar: entre 95 e 145*
> *structural_integrity: igual a 1*
> *critical_modules_status: igual a 1*
> *telemetry_link_status: igual a 1*
> *estimated_autonomy_min: mínimo de 45*

Parâmetro fixo: tempo_decolagem_min = 8

Fórmulas a serem aplicadas:
*capacity_total_kwh = (battery_voltage_v * battery_capacity_ah) / 1000*
*energy_available_kwh = capacity_total_kwh * (battery_soc_percent / 100)*
*takeoff_consumption_kwh = power_load_kw * (tempo_decolagem_min / 60)*
*real_takeoff_consumption_kwh = takeoff_consumption_kwh * (1 + energy_loss_percent / 100)*
*estimated_autonomy_min = (energy_available_kwh / (power_load_kw * (1 + energy_loss_percent / 100))) * 60*

Regra de decisão:
Definir launch_decision como READY se todos os parâmetros estiverem dentro dos limites de segurança, e como ABORT caso qualquer condição seja violada.

Observação:
O dataset deve conter tanto cenários normais quanto cenários com falhas na mesma planilha, sendo que aproximadamente 2% dos registros devem apresentar condições de falha."

A partir desse prompt, o modelo retornou a geração do dataset solicitado, conforme descrito a seguir:

"*Pronto! Criei o dataset sintético com 500 linhas, incluindo cenários normais e ~2% de falhas conforme solicitado. Também disponibilizei o arquivo CSV e o código em Python utilizado para a geração dos dados, permitindo validação e reprodutibilidade do processo.*"

E o código Python disponibilizado foi:

"*# Create synthetic telemetry dataset CSV with 500 rows*

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

np.random.seed(42)

rows = 500
fail_count = int(rows * 0.02)

start_time = datetime(2026, 1, 1, 0, 0, 0)
```

```
data = []

tempo_decolagem_min = 8


for i in range(rows):
    timestamp = start_time + timedelta(seconds=i*30)

    internal_temp = np.random.uniform(18, 35)
    external_temp = np.random.uniform(-5, 30)
    voltage = np.random.uniform(22, 30)
    current = np.random.uniform(5, 20)
    soc = np.random.uniform(60, 100)
    capacity_ah = np.random.uniform(50, 120)
    power_load = np.random.uniform(2, 8)
    energy_loss = np.random.uniform(2, 10)
    pressure = np.random.uniform(95, 145)

    structural = 1
    critical = 1
    telemetry = 1


    # Apply failure conditions to ~2%



    if i < fail_count:
        fail_type = np.random.choice(['pressure', 'telemetry', 'soc'])
        if fail_type == 'pressure':
                pressure  =  np.random.choice([np.random.uniform(80,  94),
np.random.uniform(146, 160)])
        elif fail_type == 'telemetry':
            telemetry = 0
        elif fail_type == 'soc':
            soc = np.random.uniform(30, 59)



    capacity_total_kwh = (voltage * capacity_ah) / 1000
    energy_available = capacity_total_kwh * (soc / 100)
    takeoff_consumption = power_load * (tempo_decolagem_min / 60)
     real_takeoff_consumption = takeoff_consumption * (1 + energy_loss /
100)
```

```
        estimated_autonomy = (energy_available / (power_load * (1 + energy_loss / 100))) * 60


        # Decision rule

        if (
            18 <= internal_temp <= 35 and
            -5 <= external_temp <= 30 and
            soc >= 60 and
            95 <= pressure <= 145 and
            structural == 1 and
            critical == 1 and
            telemetry == 1 and
            estimated_autonomy >= 45
        ):
            decision = "READY"

        else:
            decision = "ABORT"


        data.append([
            timestamp,
            internal_temp,
            external_temp,
            voltage,
            current,
            soc,
            capacity_ah,
            energy_available,
            power_load,
            energy_loss,
            pressure,
            structural,
            critical,
            telemetry,
            estimated_autonomy,
            decision
        ])


    columns = [
```

```
    "timestamp",
    "internal_temp_c",
    "external_temp_c",
    "battery_voltage_v",
    "battery_current_a",
    "battery_soc_percent",
    "battery_capacity_ah",
    "energy_available_kwh",
    "power_load_kw",
    "energy_loss_percent",
    "tank_pressure_bar",
    "structural_integrity",
    "critical_modules_status",
    "telemetry_link_status",
    "estimated_autonomy_min",
    "launch_decision"
]


df = pd.DataFrame(data, columns=columns)



file_path = "/mnt/data/telemetry_dataset.csv"
df.to_csv(file_path, index=False)


file_path"
```

Com base nos dados gerados, a IA também indicou padrões que podem representar comportamentos anormais no sistema. Entre as principais anomalias identificadas, destacam-se:

- Temperatura interna elevada, indicando possível superaquecimento de componentes;
- Níveis críticos de bateria, sugerindo energia insuficiente para sustentar as operações;
- Pressão fora da faixa operacional, podendo indicar falhas em válvulas ou vazamentos;
- Falha estrutural, representada pelo valor 0 na variável *structural_integrity*;
- Falhas em módulos críticos ou no sistema de telemetria, indicadas por valores 0 nas variáveis *critical_modules_status* ou *telemetry_link_status*.

Essas condições foram utilizadas para simular cenários de falha no dataset.

A partir da análise das variáveis monitoradas, foram identificados os seguintes riscos operacionais:

- **Risco térmico** — temperaturas internas elevadas podem comprometer o funcionamento dos componentes eletrônicos;
- **Risco energético** — níveis baixos de bateria podem causar perda de autonomia;
- **Risco de pressão** — valores fora da faixa podem indicar vazamentos ou sobrepressão;
- **Risco estrutural** — falhas estruturais comprometem a segurança do sistema;
- **Risco de perda de monitoramento** — falhas na telemetria dificultam a identificação de problemas em tempo real.

Dessa forma, a utilização da Inteligência Artificial permitiu gerar e analisar o dataset sintético de forma estruturada. Com base no prompt definido, foi possível simular diferentes condições operacionais e identificar anomalias e riscos, demonstrando o potencial da IA como ferramenta de apoio à análise de dados e à tomada de decisão.

6. REFLEXÃO CRÍTICA:

A utilização de algoritmos em sistemas críticos, como na exploração espacial, traz ganhos claros em precisão e rapidez na tomada de decisão. No entanto, é importante reconhecer que esses sistemas não são neutros nem infalíveis. Algoritmos apenas executam regras previamente definidas, e essas regras refletem escolhas humanas. Isso significa que qualquer erro de modelagem, simplificação excessiva ou definição inadequada de limites pode gerar decisões incorretas com consequências relevantes.

Além disso, existe um risco importante de confiança excessiva na tecnologia. À medida que os sistemas se tornam mais automatizados, pode-se assumir que o resultado gerado é sempre o mais seguro ou eficiente, quando na realidade ele depende diretamente da qualidade dos dados e da lógica implementada. Nesse sentido, a responsabilidade não é eliminada pela automação, ela é deslocada para quem projeta, valida e mantém esses sistemas.

Do ponto de vista energético, é fundamental compreender que não existe eficiência total. Conforme os princípios da termodinâmica, toda transformação de energia envolve perdas, geralmente na forma de calor. Isso significa que o objetivo não deve ser eliminar perdas o que é fisicamente impossível, mas sim reduzi-las ao máximo e, principalmente, controlá-las.

Nesse contexto, a computação tem um papel ambíguo. Por um lado, algoritmos mais eficientes, sistemas de monitorização e otimização em tempo real permitem reduzir desperdícios e melhorar o aproveitamento energético. Por outro lado, o próprio processamento de dados consome energia continuamente. Cada cálculo, cada leitura e cada decisão automatizada têm um custo energético associado. Assim, surge um ponto crítico: otimizar sistemas não pode significar apenas “usar mais tecnologia”, mas sim usar melhor.

A sustentabilidade tecnológica, portanto, não está apenas na escolha de fontes renováveis, mas na forma como a energia é utilizada. Reduzir operações desnecessárias, melhorar a eficiência dos algoritmos e projetar sistemas mais leves são decisões tão importantes quanto a geração de energia em si. Esse entendimento é reforçado pela própria área de computação, onde eficiência não é apenas desempenho, mas também consumo energético.

Do ponto de vista social, a exploração espacial levanta uma questão relevante: até que ponto o investimento em tecnologias avançadas está alinhado com as necessidades do planeta? Embora essas iniciativas impulsionam inovação e desenvolvimento científico, também exigem recursos elevados. Isso exige uma reflexão sobre prioridades e sobre como essas tecnologias podem gerar benefícios concretos para a sociedade, especialmente em áreas como energia, comunicação e monitorização ambiental.

Por fim, a tecnologia não deve ser vista como solução automática para problemas complexos. Ela amplia capacidades, mas também amplia impactos, positivos e negativos. A exploração espacial evidencia esse equilíbrio: ao mesmo tempo em que representa avanço científico, exige responsabilidade no uso de recursos, no desenvolvimento de sistemas e na tomada de decisões.

Assim, o verdadeiro desafio não está apenas em desenvolver tecnologias mais avançadas, mas em garantir que elas sejam concebidas com eficiência, responsabilidade e consciência dos seus limites físicos e impactos sociais.

No contexto deste projeto, foi possível compreender que decisões baseadas em telemetria e algoritmos impactam diretamente a segurança da missão, evidenciando a importância de sistemas bem projetados, validados e energeticamente eficientes.

7. REFERENCIAS:

GOLDEMBERG, José. *Energia e meio ambiente no Brasil*. São Paulo: Ed. Blucher, 2010.

HALLIDAY, David; RESNICK, Robert; WALKER, Jearl. *Fundamentos de Física – Termodinâmica*. 10. ed. Rio de Janeiro: LTC, 2016.

MORAN, Michael J.; SHAPIRO, Howard N. *Princípios de termodinâmica para engenharia*. 8. ed. Rio de Janeiro: LTC, 2018.

SILVA, Maurício A. *Eficiência energética e sustentabilidade ambiental*. São Paulo: Manole, 2010.